

Projet daibbar

Portfolio & Blog Personnel

Learn, build, document, and share.

Réalisé par :

DAIBBAR Mohamed
IID1 ENSA Khouribga

Encadré par :

Pr. Rabhi

Année universitaire 2025 – 2026

Table des matières

1	Introduction et Présentation du Projet	3
1.1	Qu'est-ce que daibbar ?	3
1.2	Objectifs du Projet	3
2	Architecture Technique	3
3	Structure du Projet	4
3.1	Arborescence	4
3.2	Détails du Pattern MVC	4
4	Fonctionnalités et Interface (UI/UX)	4
4.1	Design Minimaliste et Vanilla	4
4.2	Base de Données et Moteur d'Articles	5
5	API et Routage	5
6	Déploiement	5
6.1	L'approche Docker (Recommandée)	5
6.2	Variables d'Environnement	6
7	Roadmap et Perspectives	6
7.1	Version 2 (V2)	6
7.2	Version 3 (V3)	6
8	Conclusion	6

1 Introduction et Présentation du Projet

1.1 Qu'est-ce que daibbar ?

daibbar est un site web personnel doublé d'un moteur de blog léger. Son slogan officiel, *ń Learn, build, document, and share. ž* (Apprendre, construire, documenter et partager), reflète l'essence même du projet.

La page d'accueil suit un arc narratif clair et délibéré appelé **ń Proof → Knowledge → Person ž** :

- **Hero** : Une section d'accroche avec le message *Just Learn*.
- **Projets récents** : Une grille de présentation des réalisations (cartes avec des tuiles en dégradé, descriptions et technologies utilisées).
- **Derniers articles** : Un fil d'actualité sous forme de newsletter listant les dernières publications avec date, titre et aperçu.
- **About / Connect** : Une section biographique incluant des liens vers les réseaux professionnels (GitHub, LinkedIn, X, Email) agissant également comme pied de page.

1.2 Objectifs du Projet

Les principaux objectifs qui ont motivé la création de cette plateforme sont :

- **Documenter** : Centraliser l'apprentissage et archiver les différents projets réalisés.
- **Partager** : Publier des articles techniques de manière accessible.
- **Présenter** : Mettre en valeur les compétences et les réalisations techniques dans un portfolio moderne.
- **Connecter** : Faciliter la prise de contact pour d'éventuelles collaborations.

2 Architecture Technique

Le projet a été développé en adoptant une stack technique moderne, robuste et performante, principalement axée sur l'écosystème JavaScript côté serveur.

Couche	Technologie
Runtime	Node.js 20
Serveur Web	Express 5
Templates (Vues)	EJS (Embedded JavaScript)
Base de données	PostgreSQL 16
Driver DB	pg (avec gestion de pool de connexions)
Conteneurisation	Docker & Docker Compose
Frontend	CSS Vanilla, Polices système, aucun framework
Outils de développement	nodemon, dotenv

TABLE 1 – Stack technologique du projet daibbar

3 Structure du Projet

L'application suit scrupuleusement le design pattern **MVC (Modèle-Vue-Contrôleur)** pour assurer une séparation propre des responsabilités.

3.1 Arborescence

Voici la structure principale du code source :

```
1 daibbar/
2 Dockerfile           # Image pour le conteneur de l'application
3 docker-compose.yml   # Orchestration App + PostgreSQL
4 package.json
5 .env.example
6 src/
7   server.js           # Point d'entrée de l'application Express
8   db/                 # Gestion de la base de données
9     pool.js           # Pool de connexion PostgreSQL
10    populatedb.js      # Script d'initialisation de la DB (Seed)
11  routes/              # Routeurs Express
12    articles.js
13    projects.js
14  controllers/         # Logique métier et handlers de requêtes
15    ArticlesController.js
16    ProjectsController.js
17  model/               # Couche d'accès aux données
18    Article.js
19    Project.js
20  views/               # Templates EJS (Rendu côté serveur)
21    landing.ejs        # Accueil complet
22    index.ejs          # Liste des articles
23    About.ejs
24  public/              # Fichiers statiques
25    style.css           # Styles globaux
```

Structure des répertoires

3.2 Détails du Pattern MVC

- **Routes** : Elles définissent les endpoints HTTP de l'application (/articles, /projects).
- **Controllers** : Ils interceptent les requêtes, font appel aux modèles pour manipuler les données, et renvoient les vues correspondantes.
- **Model** : Encapsule les requêtes SQL via le pool **pg** pour interagir avec PostgreSQL de manière sécurisée.
- **Views** : Fichiers EJS générant le rendu HTML dynamique côté serveur.

4 Fonctionnalités et Interface (UI/UX)

4.1 Design Minimaliste et Vanilla

L'interface utilisateur a été conçue volontairement sans framework CSS volumineux. Le choix s'est porté sur du **CSS Vanilla** avec une approche esthétique très précise :

- **Palette de couleurs** : Rouge, Blanc et Noir.

- **Typographie** : Style inspiré par Apple (gros titres avec un espacement négatif généreux pour les en-têtes, *system font stack* et nombreux espaces blancs).
- **Navigation** : Barre de navigation collante (sticky) et translucide.
- Utilisation de variables personnalisées (CSS custom properties) pour gérer le thème global depuis le fichier `style.css`.

4.2 Base de Données et Moteur d'Articles

Actuellement, la base de données s'appuie sur une table principale pour stocker le contenu :

```
1 CREATE TABLE IF NOT EXISTS Article (
2   id      SERIAL PRIMARY KEY,
3   title   TEXT,
4   body    TEXT
5 );
```

Schéma de la table Article

Le script `src/db/populatedb.js` est fourni pour créer automatiquement cette table et y insérer des données de test (Seed) lors du premier déploiement.

5 API et Routage

L'application Express expose plusieurs routes pour servir les différentes pages :

Méthode	Chemin	Handler / Contrôleur	Description
GET	/	Rendu de <code>landing.ejs</code>	Affiche la page d'accueil complète.
GET	/articles	ArticlesController	Liste l'ensemble des articles issus de la base de données.
GET	/About	<i>Stub</i> (En développement)	Page "À propos".
GET	/Projects	<i>Stub</i> (En développement)	Page listant les projets.
ALL	*	404 Handler	Retourne le message d'erreur "Route existe pas".

TABLE 2 – Endpoints et Routage Express

6 Déploiement

Le projet est conçu pour être déployé très facilement grâce à la conteneurisation.

6.1 L'approche Docker (Recommandée)

L'utilisation de `docker-compose` permet de démarrer l'application Node.js (exposée sur le port 3000) ainsi que la base de données PostgreSQL (exposée sur le port 5432) de manière conjointe.

```
1 # Cloner le dépôt et configurer les variables d'environnement
2 cp .env.example .env
3
4 # Lancer les conteneurs en tâche de fond
5 docker compose up
6
7 # Initialiser la base de données (au premier lancement uniquement)
8 docker compose exec app node src/db/populatedb.js
```

Lancement via Docker

6.2 Variables d'Environnement

Un fichier `.env` à la racine du projet est nécessaire. Il inclut les valeurs de configuration suivantes :

- `DATABASE_URL` : Chaîne de connexion complète PostgreSQL (lue par `pool.js`).
- `DB_USER`, `DB_PASS`, `DB_NAME` : Paramètres utilisés pour provisionner l'instance Postgres au sein du conteneur.

7 Roadmap et Perspectives

Le projet s'inscrit dans une démarche d'amélioration continue avec plusieurs évolutions prévues.

7.1 Version 2 (V2)

- Connexion complète de la page d'accueil aux données réelles de la base de données (pour le fil d'articles).
- Implémentation complète des pages `/About` et `/Projects` avec de vraies réalisations, en remplaçant les placeholders actuels.
- Création d'une interface d'administration CRUD (Create, Read, Update, Delete) complète pour les articles.
- Mise en place d'une authentification sécurisée limitant l'accès aux actions d'administration.

7.2 Version 3 (V3)

- Ajout d'un système de mots-clés (tags) et d'un filtrage des articles par catégories.
- Intégration d'un mode Sombre/Clair dynamique (Dark Mode) en sauvegardant les préférences de l'utilisateur.

8 Conclusion

Le projet **daibbar** illustre la conception rigoureuse d'une plateforme web sur-mesure combinant un portfolio personnel et un moteur de blog léger. En s'appuyant sur des technologies back-end fiables (Node.js, Express, PostgreSQL) couplées à une approche front-end *Vanilla* minimaliste assumée, le site bénéficie d'une architecture MVC claire, facilement maintenable et déployable (via Docker). Ce rapport démontre que les bases

solides actuelles laissent place à de belles perspectives d'évolution (V2 et V3) pour enrichir continuellement ses fonctionnalités.